

MASHUP EUREKA BANK

CONTEXTO – INSTITUCIÓN BANCARIA

Se tiene una institución bancaria que ofrece servicios financieros básicos a sus clientes, tales como **consultas de movimientos, depósitos, retiros y transferencias**. Para la atención al público, el banco dispone de **cuatro ventanillas de atención (cajeros)** que operan de forma simultánea durante el horario laboral.

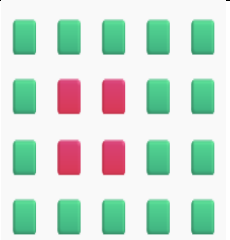
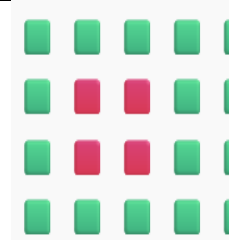
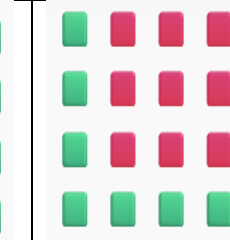
Cada cajero cuenta con un sistema informático que le permite acceder, en tiempo real, a la información de las **cuentas bancarias de los clientes**. En pantalla se muestra el estado de una cuenta específica, incluyendo el **saldo disponible**, los **movimientos recientes** y la posibilidad de ejecutar una transacción financiera.

El sistema representa el estado de cada cuenta de la siguiente manera:

- **Cuenta disponible (libre):** se muestra en color **verde**, indicando que puede ser utilizada para realizar una transacción.
- **Cuenta en proceso de transacción (bloqueada):** se muestra en color **rojo**, indicando que otro cajero está operando sobre esa misma cuenta y no puede ser utilizada temporalmente.

ESCENARIO DE CONCURRENCIA

Suponga que **tres clientes distintos** acuden al banco y solicitan realizar operaciones sobre la **misma cuenta bancaria** (por ejemplo, una transferencia, un retiro y una consulta de movimientos) en el mismo momento y en diferentes ventanillas. Además, un **cuarto cliente** realiza una transacción sobre una **cuenta diferente**.

CAJERO 1	CAJERO 2	CAJERO 3	CAJERO 4
			
CUENTA 1	CUENTA 1	CUENTA 1	CUENTA 2

INTERPRETACIÓN

- Los **cajeros 1, 2 y 3** visualizan **exactamente el mismo estado** porque trabajan sobre la **misma cuenta bancaria**.
- Las operaciones marcadas en **rojo** están **bloqueadas**, ya que una transacción está en ejecución.
- El **cajero 4** trabaja sobre **otra cuenta**, por lo que su estado es **independiente**.

- El sistema garantiza:
 - Exclusión mutua
 - Consistencia
 - Actualización automática en tiempo real

Ante esta situación:

- El sistema debe permitir que solo **una operación a la vez** se ejecute sobre una cuenta bancaria específica.
- Cuando un cajero inicia una transacción sobre una cuenta, esta debe **bloquearse automáticamente** para los demás cajeros.
- Las pantallas de los otros cajeros deben **actualizarse en tiempo real**, mostrando la cuenta como ocupada (color rojo).
- Las operaciones sobre cuentas distintas deben poder ejecutarse **de manera paralela**, sin interferencias.

CASO PRÁCTICO PARA DESARROLLAR

Se pide desarrollar una **propuesta de solución** que simule la interacción de los sistemas de ventanilla del banco para la gestión de transacciones financieras.

Si un usuario (cajero o cliente mediante autoservicio) inicia una operación sobre una cuenta bancaria, el sistema debe:

1. **Notificar automáticamente** a las demás ventanillas que dicha cuenta se encuentra en uso.
2. **Bloquear la cuenta** durante la ejecución de la transacción, evitando que otro usuario realice operaciones simultáneas sobre la misma.
3. **Actualizar el estado de la cuenta en tiempo real** en todas las pantallas del sistema.
4. **Liberar la cuenta** una vez finalizada la transacción, permitiendo nuevas operaciones.

El objetivo es **automatizar y estandarizar los procesos operativos del banco**, garantizando:

- Integridad de los datos.
- Consistencia de las transacciones.
- Correcta sincronización entre múltiples usuarios concurrentes.

ENTREGABLES:

- Transformación del caso de estudio a historias de usuario.
- Diagrama de clases, diagrama de despliegue y diagrama de componentes de la solución propuesta utilizando UML.
- Código fuente desarrollado en cualquier lenguaje de programación orientado a objetos, ya sea para uso mediante aplicación Web o Móvil. La utilización de bases de datos es opcional y no será evaluada.
- Implementación de al menos una prueba unitaria.

En su Proyecto además de la arquitectura SOA a implementar deberán existir los CRUD necesarios para cumplir con los planteamientos requeridos.

PREGUNTAS A RESPONDER:

1. Justifique la metodología de desarrollo de software utilizada para dar solución al caso planteado.
2. Indique cuál fue la arquitectura de software propuesta como solución, justifique su respuesta considerando criterios de calidad de software como seguridad, fiabilidad, usabilidad, entre otros.
3. En función de la solución planteada indique cuáles fueron los patrones de diseño de software utilizados, justificando su respuesta.
4. Justifique la prueba unitaria planteada para la validación y verificación del producto de software.

AYUDA PARA LAS PRUEBAS

Para el desarrollo de software, es crucial realizar varias pruebas para garantizar la calidad y funcionalidad del producto. Aquí tienes algunos tipos de pruebas comunes:

1. **Pruebas Unitarias (Unit Testing):**
 - Se centran en probar componentes individuales o funciones del código de manera aislada.
 - Ayudan a verificar que cada parte del software funcione correctamente de forma independiente.
2. **Pruebas de Integración (Integration Testing):**
 - Se enfocan en probar la interacción entre diferentes módulos o componentes del sistema.
 - Aseguran que los componentes integrados funcionen juntos como se espera.
3. **Pruebas de Sistema (System Testing):**
 - Evalúan el sistema completo para asegurar que cumple con los requisitos especificados.
 - Incluyen pruebas funcionales y no funcionales.
4. **Pruebas de Regresión (Regression Testing):**
 - Se realizan para asegurarse de que los cambios recientes en el código no hayan introducido nuevos errores.
 - Es importante realizar estas pruebas después de cualquier modificación o actualización.
5. **Pruebas de Aceptación (Acceptance Testing):**
 - Se llevan a cabo para verificar si el sistema cumple con los requisitos del usuario y si es apto para su uso.
 - Estas pruebas suelen ser realizadas por los usuarios finales o stakeholders.
6. **Pruebas de Rendimiento (Performance Testing):**
 - Evalúan el rendimiento del sistema bajo diferentes condiciones, como la carga y el estrés.
 - Incluyen pruebas de carga, pruebas de estrés y pruebas de escalabilidad.
7. **Pruebas de Seguridad (Security Testing):**
 - Verifican la seguridad del sistema y aseguran que los datos y la funcionalidad estén protegidos contra ataques.
8. **Pruebas de Usabilidad (Usability Testing):**
 - Evalúan la facilidad de uso del sistema y la experiencia del usuario.
 - Pueden incluir estudios de usuarios y análisis de la interfaz.
9. **Pruebas de Interfaz de Usuario (UI Testing):**
 - Se enfocan en la apariencia y el comportamiento de la interfaz de usuario.
 - Verifican que los elementos visuales se muestren y funcionen correctamente.
10. **Pruebas de Compatibilidad (Compatibility Testing):**
 - Aseguran que el software funcione correctamente en diferentes navegadores, dispositivos y sistemas operativos.

Cada tipo de prueba tiene su propio propósito y es importante utilizarlas de manera adecuada para asegurar un software de alta calidad.